

ADR'S

SISTEMAS DISTRIBUIDOS

Sergio Luis Angel Romero

DOCENTE: JESUS ARIEL

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA

CORPORACION UNIVERSITARIA DEL HUILA
Vignada Minieducación

ING. EN SISTEMAS

NEIVA - HUILA

2026

Tabla de Contenido

Tabla de Contenido	2
1. Descripción del Proyecto.....	4
2. Listado de ADR Identificados.....	4
ADR-001 – Descuento Automático de Stock	4
SLAR-HU-014	4
Identificación del patrón	4
Análisis del proyecto	5
Antipatrón identificado	5
Segmento de código del proyecto	5
Diseño e implementación del ADR.....	6
Impacto sobre el sistema	6
ADR-002 – Seguimiento de Pedido con Línea de Tiempo.....	7
SLAR-HU-015	7
Identificación del patrón	7
Análisis del proyecto	7
Antipatrón identificado	7
Segmento de código del proyecto	8
Diseño e implementación del ADR.....	8
Impacto sobre el sistema	8
ADR-003 – Reportes PDF Descargables.....	9
SLAR-HU-016	9
Identificación del patrón	9
Análisis del proyecto	9
Antipatrón identificado	9
Segmento de código del proyecto	10
Diseño e implementación del ADR.....	10
Impacto sobre el sistema	11



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

1. Descripción del Proyecto

El sistema E-Commerce es una aplicación full-stack desarrollada con Spring Boot en el backend y React en el frontend. Permite gestionar productos, pedidos, carritos de compra y pagos mediante una API REST. Durante el sprint documentado se implementaron tres funcionalidades clave: descuento automático de stock, seguimiento visual de pedidos y generación de reportes PDF descargables.

2. Listado de ADR Identificados

ADR-001 – Descuento Automático de Stock al Completar Pago

ADR-002 – Seguimiento de Pedido con Línea de Tiempo Visual

ADR-003 – Generación y Descarga de Reportes en Formato PDF

ADR-001 – Descuento Automático de Stock

ID	ADR-001
Título	Descuento Automático de Stock al Completar Pago
Estado	✓ Implementado
Fecha	Mayo 2026
Categoría	Lógica de Negocio / Gestión de Inventario

SLAR-HU-014

Como administrador del sistema, quiero que el inventario se actualice automáticamente al completarse un pago, para mantener el stock real sin intervención manual.

Identificación del patrón

Se implementa el patrón de Transacción Atómica combinado con el patrón de Estado (State Pattern) para el carrito. El stock se descuenta en el mismo contexto transaccional del pago, garantizando consistencia de datos. El estado intermedio PROCESANDO actúa como cerrojo optimista ante pagos concurrentes.

Análisis del proyecto

El sistema de e-commerce requiere una gestión de inventario confiable y sin fricción. Antes de esta implementación, el stock no se descontaba de forma automática al realizarse una compra. Esto implicaba riesgo de overselling, inconsistencias de datos y necesidad de intervención manual del administrador. El sistema necesitaba:

- Garantizar consistencia del stock en tiempo real.
- Manejar correctamente los flujos de pago y reembolso.
- Permitir identificar con precisión qué productos descontar por cada transacción.
- Registrar trazabilidad de todos los cambios de inventario.

Antipatrón identificado

El antipatrón sería descontar el stock en un paso separado del pago, sin control transaccional. Esto genera condiciones de carrera: dos usuarios podrían comprar el último ítem simultáneamente si el descuento no es atómico con la confirmación del pago.

Problemas del antipatrón:

- Stock desactualizado si el pago falla después del descuento.
- Riesgo de overselling ante accesos concurrentes.
- Necesidad de reconciliación manual del inventario.

Segmento de código del proyecto

PagoService.procesarPago() – descuento al completar pago:

```
CarritoEntity carrito = carritoRepo.findByClienteAndEstado(cliente,
PROCESANDO);
carrito.getItems().forEach(item -> {
    producto.setStock(producto.getStock() - item.getCantidad());
    auditoriaService.log("DESCUENTO", producto, item.getCantidad());
});
```

PagoService.reembolsarPago() – restauración de stock:

```
carrito.getItems().forEach(item -> {
    producto.setStock(producto.getStock() + item.getCantidad());
    auditoriaService.log("RESTAURACION", producto, item.getCantidad());
});
```

CarritoService.iniciarCheckout() – nuevo método:

```
carrito.setEstado(EstadoCarrito.PROCESANDO);
carritoRepo.save(carrito);
```

CarritoController – nuevo endpoint:

```
POST /carrito/cliente/{id}/checkout
```

api.js – nueva función frontend:

```
export const iniciarCheckout = (clienteId) =>
    api.post(`/carrito/cliente/${clienteId}/checkout`);
```

Diseño e implementación del ADR

Se diseñó un flujo de tres etapas para el carrito:

ACTIVO → PROCESANDO → COMPLETADO (o REEMBOLSADO)

El método `iniciarCheckout()` transicióna el carrito a PROCESANDO antes del pago. El método `procesarPago()` busca el carrito en estado PROCESANDO para identificar exactamente qué ítems descontar. La auditoría registra cada movimiento de stock (descuento y restauración) con timestamp y referencia al pedido.

Impacto sobre el sistema

Beneficios:

- Stock actualizado en tiempo real, eliminando riesgo de overselling.
- Flujo de reembolso coherente con restauración automática de inventario.
- Auditoría completa de todos los movimientos de stock.
- Estado PROCESANDO como cerrojo optimista ante pagos concurrentes.

ACTIVO	→	PROCESANDO	→	COMPLETADO	↺	REEMBOLSO
Cliente agrega ítems		<code>iniciarCheckout()</code>		<code>procesarPago()</code> → stock --		stock ++

Consideraciones:

- Se debe garantizar atomicidad de la transacción pago-descuento para evitar inconsistencias ante fallos.
- El estado PROCESANDO debe tener tiempo de expiración para no bloquear carritos ante pagos fallidos.

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

ADR-002 – Seguimiento de Pedido con Línea de Tiempo

ID	ADR-002
Título	Seguimiento Visual de Pedido mediante Línea de Tiempo
Estado	✓ Implementado
Fecha	Mayo 2026
Categoría	Experiencia de Usuario / Gestión de Estados

SLAR-HU-015

Como usuario comprador, quiero ver el estado de mi pedido en una línea de tiempo visual, para saber en qué etapa se encuentra mi compra.

Identificación del patrón

Se implementa el patrón State Machine (máquina de estados) en el backend con un motor de transiciones explícito, y el patrón de componente visual Timeline en el frontend. El motor de transiciones previene flujos inválidos validando cada avance de estado antes de persistirlo.

Análisis del proyecto

Los usuarios necesitaban visibilidad sobre el estado de sus pedidos tras completar una compra. El sistema original carecía de estados de entrega (EN_CAMINO, ENTREGADO) y de una interfaz visual que mostrara el progreso del pedido de forma clara e intuitiva. Se requirió:

- Ampliar el modelo de estados del pedido para cubrir el ciclo completo de entrega.
- Validar las transiciones de estado para prevenir flujos inválidos.
- Ofrecer al comprador una experiencia visual clara del progreso de su pedido.
- Dotar al administrador de controles para avanzar el estado manualmente.

Antipatrón identificado

Un antipatrón sería permitir transiciones libres de estado sin validación, o mostrar el estado solo como texto plano sin contexto visual. Problemas:

- Un pedido podría pasar de CREADO directamente a ENTREGADO omitiendo etapas.
- El usuario no podría distinguir en qué etapa real se encuentra su pedido.
- El administrador podría generar estados incoherentes en la base de datos.

Segmento de código del proyecto

Backend – Nuevos estados en EstadoPedido:

```
EN_CAMINO,    // Pedido despachado y en tránsito
ENTREGADO     // Pedido recibido por el cliente
```

PedidoService.TRANSICIONES – nuevas transiciones válidas:

```
PAGADO        → EN_CAMINO
EN_CAMINO     → ENTREGADO
```

Changeset 010-nuevos-estados.xml – actualización del trigger:

```
ALTER FUNCTION fn validar_transicion_estado
-- Incluye EN_CAMINO y ENTREGADO en la lógica de validación
```

Frontend – TrackingTimeline en MisPedidos.jsx:

Componente visual con 5 pasos: Creado → Confirmado → Pago recibido → En camino → Entregado

```
const pasos = ['CREADO', 'CONFIRMADO', 'PAGADO', 'EN_CAMINO', 'ENTREGADO'];
{pasos.map((paso, i) => (
  <Step key={i} active={estadoActual === paso} label={paso} />
))}
```

AdminPedidos.jsx – controles de avance de estado:

```
// Botones PAGADO → EN_CAMINO → ENTREGADO con iconos y colores
<Button onClick={() => avanzarEstado(pedidoId, 'EN_CAMINO')}>
  Despachar pedido
</Button>
```

Diseño e implementación del ADR

Se amplió el enum EstadoPedido con EN_CAMINO y ENTREGADO. El mapa TRANSICIONES en PedidoService define las únicas rutas válidas. El changeset Liquibase 010 actualizó el trigger de base de datos. En el frontend, TrackingTimeline renderiza los 5 pasos con animación pulsante en el paso activo y badges especiales para CANCELADO y REEMBOLSADO.

Impacto sobre el sistema

Beneficios:

- Experiencia de usuario mejorada con visibilidad completa del ciclo de entrega.
- Motor de transiciones robusto que previene estados inválidos.
- Migración versionada que garantiza consistencia en todos los entornos.
- Panel administrativo con control granular sobre el avance de pedidos.

Consideraciones:

- Las transiciones son manejadas manualmente por el administrador; futuras integraciones con logística podrían automatizarlas.
- El componente TrackingTimeline asume los 5 pasos definidos; agregar nuevos estados requiere actualizar el componente.

ADR-003 – Reportes PDF Descargables

ID	ADR-002
Título	Seguimiento Visual de Pedido mediante Línea de Tiempo
Estado	✅ Implementado
Fecha	Mayo 2026
Categoría	Experiencia de Usuario / Gestión de Estados

SLAR-HU-016

Como administrador, quiero descargar reportes en PDF de ventas e inventario, para analizar el rendimiento del negocio y presentar informes.

Identificación del patrón

Se implementa el patrón de Generación de Documentos bajo demanda (Document Generation On-Demand) con separación clara entre lógica de datos (ReporteService) y exposición HTTP (ReporteController). El frontend descarga el binario mediante responseType blob sin almacenarlo en el servidor.

Análisis del proyecto

El sistema carecía de capacidad de reporting exportable. Los administradores necesitaban informes formales de ventas e inventario en un formato portable y listo para presentar. Se requirió:

- Generación de reportes de ventas filtrados por rango de fechas.
- Generación de reportes de inventario con alertas visuales de stock bajo.
- Exportación en formato PDF, estándar para reportes empresariales.
- Integración fluida con el dashboard administrativo existente.

Antipatrón identificado

Un antipatrón sería generar los reportes en el frontend con datos crudos del API, o guardar los PDFs en el servidor y servirlos como archivos estáticos. Problemas:

- Reportes generados en cliente implican enviar todos los datos al navegador.
- PDFs almacenados en servidor consumen almacenamiento y pueden quedar obsoletos.
- Falta de control sobre el formato y contenido del reporte desde el servidor.

Segmento de código del proyecto

Dependencia agregada al pom.xml:

```
<dependency>
  <groupId>com.itextpdf</groupId>
  <artifactId>itext7-core</artifactId>
  <version>7.2.5</version>
</dependency>
```

ReporteService – métodos de generación:

```
public byte[] generarReporteVentas(LocalDate desde, LocalDate hasta) {
    // Tabla de pedidos + resumen por estado + total facturado
}
public byte[] generarReporteProductos() {
    // Inventario completo + productos sin stock en rojo
}
```

ReporteController – endpoints REST:

```
GET /reportes/ventas?desde=YYYY-MM-DD&hasta=YYYY-MM-DD
GET /reportes/productos
// Ambos devuelven application/pdf
// Content-Disposition: attachment
```

vite.config.js – proxy configurado:

```
'/reportes': { target: 'http://localhost:8080' }
```

api.js – funciones de descarga en frontend:

```
export const descargarReporteVentas = (desde, hasta) =>
  api.get(`/reportes/ventas?desde=${desde}&hasta=${hasta}`,
    { responseType: 'blob' });
export const descargarReporteProductos = () =>
  api.get('/reportes/productos', { responseType: 'blob' });
```

Diseño e implementación del ADR

Se eligió iText7 como librería de generación de PDFs en el backend por su madurez, flexibilidad y soporte activo en el ecosistema Java. Los endpoints devuelven el PDF como adjunto descargable (Content-Disposition: attachment). En el frontend, los botones del AdminDashboard usan responseType blob para manejar la respuesta binaria y forzar la descarga en el navegador.

Ambos botones muestran el estado de carga (Generando...) mientras se procesa la solicitud.

Impacto sobre el sistema

Beneficios:

- Reportes exportables en formato estándar listos para presentaciones y archivado.
- iText7 permite reportes con formato profesional, tablas y estilos visuales.
- Arquitectura REST limpia con separación clara entre generación (backend) y descarga (frontend).
- Reportes generados bajo demanda, siempre con datos actuales.

Reporte de Ventas (PDF)	Reporte de Inventario (PDF)
Descarga el reporte del mes actual. Muestra estado de carga mientras se genera.	Descarga el inventario actual completo. Muestra estado de carga mientras se genera.

Consideraciones:

- La generación de PDFs grandes puede impactar el tiempo de respuesta; se recomienda evaluar generación asíncrona para períodos extensos.
- La dependencia iText7 (AGPL) debe revisarse ante cambios de licenciamiento o distribución del sistema.

Cambios e implementación observados en ;

Repos: <https://github.com/sangel3232/Gestor-de-Pedidos-Portal/tree/SLAR-HU-014>

<https://github.com/sangel3232/Gestor-de-Pedidos-Api/tree/SLAR-HU-015>

<https://github.com/sangel3232/Gestor-de-Pedidos-Portal/tree/SLAR-HU-016>